BrickMore

Anonymous submission

Abstract

Generating brick structures from text is a geometric assembly problem: a layout must match a natural-language description while placing discrete pieces that fit in a bounded grid, connect through studs, and form one consistent structure. We present BrickMore, a text-to-brick framework that combines agentic batch generation of training data, language-model fine-tuning, and post-generation geometric repair in order to learn and constrain a spatial representation of brick assemblies. Using GPT-6.1 Sol, we run an agentic batch-generation pipeline in which the model writes parametric generator programs that then emit three-dimensional voxel models in batches. The resulting BrickMore-48 source dataset contains 80,000 voxel models across 21 object categories on a 48^3 grid, each with five bilingual captions at different levels of detail (400,000 records). Same-color tiling, connectivity analysis, and local bridging and retiling convert the shapes into brick layouts; in the latest integrated release, 79,301 models pass the specified geometric checks. We fine-tune Qwen3-1.7B with low-rank adaptation (LoRA) on English captions and brick-placement sequences so that it predicts brick dimensions and positions from text. After generation, the pipeline forms a voxel union from the in-bounds portions of supported placements, uses mixed-integer linear programming (MILP) to seek an exact cover, and checks for a single stud-connected component. On 114 shared prompts and under a common rendering protocol, the evaluated BrickMore outputs (110 retiled, 4 raw) obtain the highest mean scores on three text–image alignment metrics, and a mean invalid-placement ratio of 0.00%, matching BrickGPT.

1 Introduction

Text-to-3D generation lets users describe and create objects in natural language (Poole et al. 2023). Generating brick structures adds the requirement that a target shape be represented as discrete pieces and their spatial placements. Beyond matching the text, the structure must avoid overlaps and keep the bricks connected: a layout may resemble the described object while containing intersecting bricks or disconnected parts. The setting is therefore one of geometric reasoning and object assembly, in which the system must decide how bricks fit, interlock, and remain connected in space, not only whether the overall shape resembles the described object.

Prior work has explored text-driven brick generation with different representations and constraints. BrickGPT trains an autoregressive language model on paired text and brick structures and applies validity checks and physics-aware rollback during inference (Pun et al. 2025). BrickNet uses a graph-backed representation to generate assembly sequences over a broader range of parts and connection types (Kulits and Schmid 2026). Both show that structural representations and generation-time constraints matter for brick modeling.

For voxel-based shapes, spatial adjacency between voxels does not imply a valid connection between bricks: a voxel shape can form one connected volume even when the bricks covering it only touch side by side without stud connections. This distinction affects both data construction and generation. Converting voxel shapes into training targets requires attention to coverage and stud connectivity, and predicted layouts may contain overlaps or disconnected components that require further processing.

We present BrickMore to study this problem. With GPT-6.1 Sol, we develop an agentic batch-generation pipeline: the model writes parametric generator programs, the programs emit voxel models in batches, and we convert the models into brick layouts. The source dataset contains 80,000 models across 21 categories on a 48^3 grid, each with five bilingual captions. Occupied voxels are tiled with same-color rectangular bricks, and connection analysis with local bridging and retiling addresses ungrounded components; limited changes to the source voxels are recorded separately. In the latest integrated release, 79,301 models pass the specified geometric checks.

We fine-tune Qwen3-1.7B (Yang et al. 2025) with LoRA (Hu et al. 2022) on English captions and brick-placement sequences. Given a prompt, the model predicts brick dimensions and positions; the current output does not include color. To handle overlaps and connectivity defects, we form a target union from the in-bounds voxels of supported placements, use MILP to seek an exact cover, and check for a single stud-connected component; when no cover is found within the solver budget, a new sampling seed can be queued. The 48^3 workspace, multi-level captions, and fixed spatial serialization define what the model learns, while conversion-time checks and post-generation

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

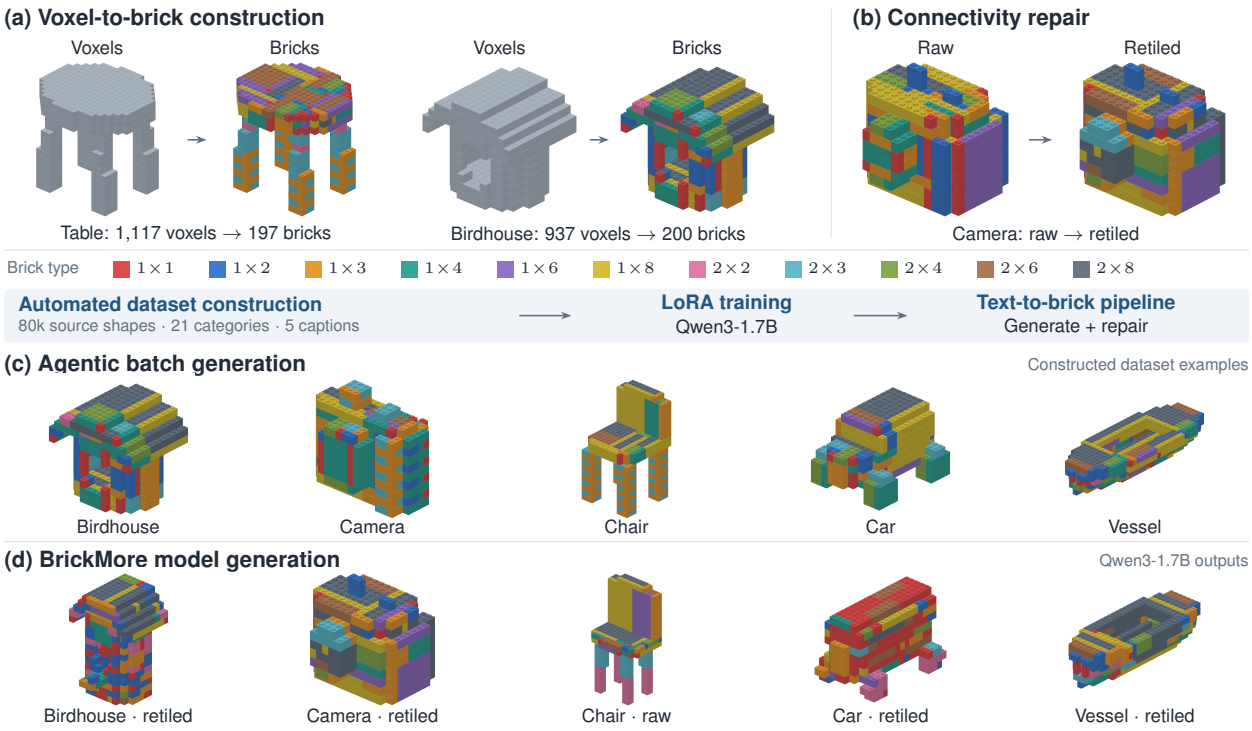


Figure 1: **BrickMore overview.** (a) Source voxels are converted into rectangular bricks. (b) Raw predictions are retiled to repair connectivity. (c) Examples from agentic batch generation of voxel data. (d) Outputs of the fine-tuned Qwen3-1.7B pipeline. Colors encode brick type independent of orientation; they are visualization labels, not predicted appearance. Voxels are shown in gray.

exact-cover retiling constrain coverage, overlaps, and stud connectivity. The connectivity check is geometric; physical stability and assembly-order feasibility require separate evaluation.

We compare the provided outputs of BrickMore, BrickGPT, and BrickNet on 114 shared prompts spanning 21 categories and five description levels, measuring text–image alignment with VQAScore, PE, and SigLIP2 and the invalid-placement ratio P_{inv}. Under a common rendering protocol, BrickMore has the highest mean on all three alignment metrics, and BrickMore and BrickGPT both have a mean P_{inv} of 0.00% in exported file order. These results describe the provided outputs under the stated protocol and do not by themselves establish physical stability or assembly feasibility.

Our contributions are as follows.

1. An automated 48^3 voxel data-generation pipeline based on agentic batch generation with GPT-6.1 Sol, which writes parametric generator programs that produce 80,000 source models across 21 categories with five bilingual captions each; 79,301 models pass the specified geometric checks in the latest integrated release. The data supply geometrically checked layouts for fine-tuning a text-conditioned spatial representation of brick assemblies.
2. A voxel-to-brick conversion pipeline that combines same-color tiling, connection analysis, and local bridging and retiling, and records limited changes to source voxels, so that training targets encode stud-connected assemblies rather than voxel adjacency alone.
3. A fine-tuned Qwen3-1.7B text-to-brick generator combined with post-generation repair based on voxel unions, exact-cover MILP, and connectivity checks.

2 Related Work

Brick assembly and generation. Learning-based brick construction has been studied as graph generation (Thompson et al. 2020), as combinatorial construction with deep reinforcement learning (Chung et al. 2021), and as interactive structural understanding in a LEGO simulator (Walsman et al. 2022). For text-conditioned generation, BrickGPT represents a design as an autoregressively generated sequence of brick placements and uses brick-level rejection sampling with physics-aware rollback at inference (Pun et al. 2025); BrickNet represents part connectivity through a graph-backed parametrization and trains autoregressive models on its assembly sequences (Kulits and Schmid 2026). BrickMore also generates placement sequences from text, on a larger 48^3 grid with a 20-type library, and examines geometric reconstruction and connectivity in a separate post-generation stage.

Stability analysis. StableLego analyzes the structural stability of block-stacking assemblies with a force-balance optimization (Liu et al. 2024). Such analyses depend on their

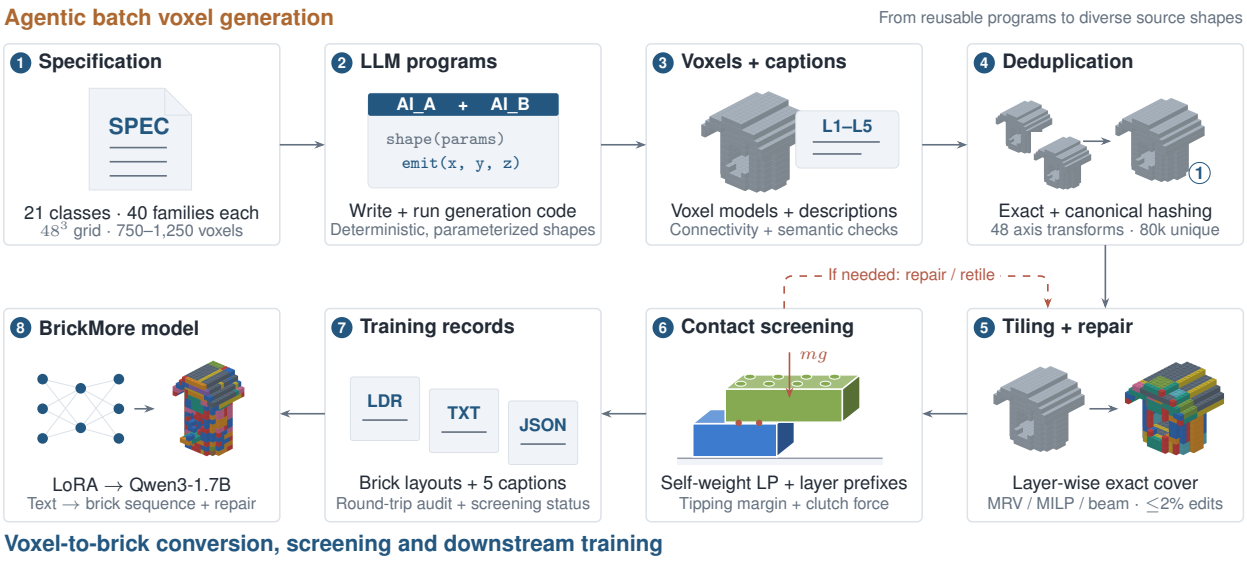


Figure 2: **Data-construction workflow.** Parametric programs written by GPT-6.1 Sol through agentic batch generation produce voxel shapes and captions, followed by deduplication, voxel-to-brick tiling, repair, contact-based screening, and audited export for fine-tuning. The dashed arrow denotes repair or retiling when needed. Screening is distinct from a guarantee of physical stability. Brick colors follow the legend in Figure 1.

mechanical assumptions. In BrickMore, geometric connectivity and the contact-based screening in the dataset materials address different properties; neither alone establishes that every generated model can be physically assembled and remains stable.

Procedural data and parameter-efficient fine-tuning. Procedural generation is a common way to obtain large, controllable 3D training data (Raistrick et al. 2023). BrickMore uses a language model to write the procedural generators rather than to emit coordinates directly. The generator itself is adapted with LoRA (Hu et al. 2022), following the BrickGPT recipe of fine-tuning a pretrained language model on serialized brick sequences (Pun et al. 2025).

3 Dataset Construction

3.1 Parametric Voxel Generation

BrickMore-48 contains 80,000 voxel models on a 48^3 grid across 21 categories (3,805–3,810 per category), each paired with five bilingual captions at levels L1 (concise) to L5 (detailed). The models are produced by parametric generator programs written through agentic batch generation with GPT-6.1 Sol; the programs, rather than the language model itself, emit the voxel coordinates. Each record includes a geometry plan and a dataset version identifier for reproduction, and exact and canonical (48 axis/sign transforms) occupancy hashes show no duplicates across the two 40,000-model shards. Figure 2 summarizes the workflow; sample shapes are shown in Appendix A.

3.2 Voxel-to-Brick Conversion and Checks

The conversion pipeline tiles occupied voxels with same-color rectangular bricks and analyzes their connections. Local bridging and retiling address ungrounded components, and changes to source voxels are recorded separately. Each converted model carries a `validation.json` that records three geometric constraints: the occupied voxels form a single six-neighbour connected component, every brick component is grounded, and the bricks exactly cover the (derived) target voxels. Under these release rules a layout may consist of several brick components that are each grounded; this criterion differs from the single stud-connected component enforced by post-generation retiling (Section 4). The dataset materials also describe a contact-based gravity screening procedure; its outputs are screening results under stated assumptions rather than proof of physical stability (Appendix A); training-data selection uses the geometric checks.

3.3 Integrated Release and Training Corpus

The full retile release records 79,298 geometrically passing models; the 702 failures (0.88%) are concentrated in table (132), bookshelf (110), vessel (89), bowl (79), and bus (75). Replacing all 1,905 batch-B piano slots (1,902 passing) with a regenerated release in which all 1,905 pass gives an integrated count of $79{,}298 - 1{,}902 + 1{,}905 = 79{,}301$ passing models. This number is computed from the two release reports, not from a new full-dataset verification run, and the original A/B download does not apply the replacement automatically. Fine-tuning uses only geometrically passing models (Table 1).

4 BrickMore Text-to-Brick Model

4.1 Fine-Tuning

We fine-tune Qwen3-1.7B on BrickMore-48 with a library of 20 brick types. Relative to StableText2Brick used by BrickGPT (20^3, 14 types), the library adds six footprints (3×2,

Item	Count
Source models / categories / grid	80,000 / 21 / 48^3
Bilingual caption records	400,000
Failing in full retile release	702 (0.88%)
Passing in full retile release	79,298
Piano models restored by revised release	3
Integrated passing models	79,301
(model, English caption) samples	396,610
Training / validation samples	356,955 / 39,655
Tokens over 3 epochs	2,228,044,818

Table 1: Dataset and fine-tuning corpus.

Item	Value
Base model	Qwen3-1.7B (bf16)
Adaptation	LoRA, $r = 16$, $\alpha = 32$, dropout 0.05, all linear layers
Optimization	LR 10^{-4}, cosine, 100 warm-up steps, 3 epochs
Batch / max length	effective 8 (8 GPUs $\times$ 1) / 8,192
Hardware	DDP, 8×RTX 4090D (24 GB)
Released checkpoint	step 79,000 (best validation loss)
Software	transformers 4.51.3, trl 0.17.0, peft 0.15.2

Table 2: Fine-tuning configuration (details in Appendix B).

2×3, 3×1, 1×3, 2×8, 8×2). Each object occupies 750–1,250 voxels with coordinates in $\{0, \ldots, 47\}$. Training uses only the English captions (`captionEn`, L1–L5); the Chinese captions are not used. Samples are split by model at a 9:1 ratio, so the captions of one model never straddle the split.

Each sample is a three-turn chat, structurally identical to the official BrickGPT data-preparation script: a system prompt ("You are a helpful assistant."), a user instruction adapted to the 48^3 world that lists the 20 allowed dimensions, and an assistant reply with one brick per line, e.g. 2×4 (x, y, z). No thinking content is used; inference sets `enable_thinking=False` to match training. Brick dimensions are serialized Y-first, then X, which differs from the StableText2Brick convention; training and inference share this convention, and downstream parsers must restore the orientation consistently.

Hyperparameters follow the Qwen3 fine-tuning script in the official BrickGPT repository (Table 2) rather than the LLaMA numbers in the original paper; Appendix B compares the two. Sequences are unpacked so that each caption–sequence boundary is preserved, and dependencies are pinned to the official TRL 0.17 stack. Training runs 133,860 steps in 25.3 hours (≈202 GPU-hours). Validation loss, computed every 500 steps on a 2,000-sample validation subset, reaches its minimum of 0.0308 at step 79,000 (epoch 1.77) and rises to 0.0339 by the end of epoch 3. Because `load_best_model_at_end` is enabled, the released weights are those of step 79,000, so the additional third epoch does not degrade the released model. Full loss curves are listed in Appendix B.

Algorithm 1 BrickMore inference and retiling

Require: prompt c; fine-tuned model θ; library $\mathcal{L}$ (20 types); seeds $S = (42, 104, 2026)$; MILP tiers $T = (12, 60, 180)$ s
Ensure: $(B_{\text{out}}, \textit{status})$, $\textit{status} \in \{\textsc{Success}, \textsc{Draft}, \textsc{Failed}\}$

$B \leftarrow \emptyset$; $A \leftarrow \emptyset$
for $s \in S$ **do**
  $(B_s, \textit{rep}) \leftarrow \mathrm{Parse}(\theta.\mathrm{Generate}(c,\ \mathrm{seed}{=}s))$
  $A \leftarrow A \cup \{(B_s, \textit{rep})\}$
  if $\textit{rep}.\mathrm{complete} \wedge \textit{rep}.\mathrm{valid}$ **then**
    $B \leftarrow B_s$; **break**
  end if
end for
if $B = \emptyset$ **then**
  if no completed draft in A **then**
    return $(\emptyset,\ \textsc{Failed})$
  end if
  $B \leftarrow \arg\min_{\text{completed } a \in A} \mathrm{Err}(a)$
end if
$C \leftarrow \mathrm{UnionVoxels}(B;\ \mathcal{L},\ 48^3)$ {clip; overlaps count once}
for $\tau \in T$ **do**
  $B^* \leftarrow \text{MILP-SingleStudCover}(C,\ \tau)$
  if $B^* \neq \emptyset$ **then**
    $\mathrm{Audit}(B^*, C)$ {one stud component, voxels(B^*)=C}
    return $(B^*,\ \textsc{Success})$
  end if
end for
$\mathrm{QueueReseed}(c)$
return $(B,\ \textsc{Draft})$ {unsolved: keep raw draft}

4.2 Inference and Retiling

Inference follows the seed-escalation and repair procedure in Algorithm 1. The model samples a placement sequence with thinking disabled, temperature 0.7, top-p 0.9, and at most 8192 new tokens, using seeds 42, 104, and 2026 in order. A candidate is accepted when generation completes (ends with an end-of-sequence token) and the geometry validator reports no errors. Otherwise, the pipeline keeps the completed draft with the fewest validator error records $\mathrm{Err}(\cdot)$ (line format, library membership, bounds, and overlaps). It then forms the voxel union C of the in-bounds cells of supported library placements, solves a shape-preserving exact cover of C with a single stud-connected component by MILP under a 12–60–180 s time ladder, and audits the result independently. If no cover is found within this budget, the raw draft is retained and a new seed may be queued. A separate recovery path may extend the budget to 600 s and sample seeds outside S; it is not part of the default loop. We call the MILP output the *retiled* layout (files are named `stitched`) and the model's draft the *raw* layout.

5 Comparative Evaluation

5.1 Setup and Metrics

We compare the provided outputs of BrickMore, BrickGPT, and BrickNet on 114 shared prompts covering 21 categories and five description levels (one output per system and prompt). For BrickMore, the provided output is the first candidate, using the retiled layout when available: 110 of the 114 outputs are retiled and 4 are raw. All systems are rendered with the same protocol: uniform yellow material, 512×512

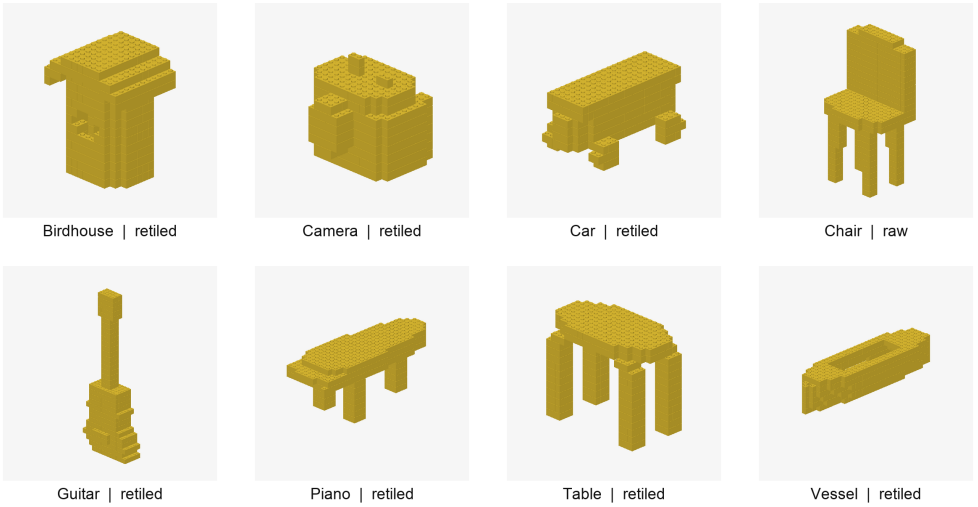


Figure 3: Selected BrickMore outputs from the 114-prompt evaluation, rendered with the uniform evaluation protocol (not photographs of physical assemblies). In this gallery, one chair is raw and the other outputs are retiled; in the full main comparison, 4 of the 114 BrickMore outputs are raw.

System	$P_{\text{inv}} \downarrow$	VQA ↑	PE ↑	SigLIP2 ↑
BrickMore	0.00%	**0.2176**	**0.1856**	**0.1231**
BrickGPT	0.00%	0.1401	0.1457	0.0297
BrickNet	21.99%	0.0624	0.1302	0.0100

Table 3: Main comparison on 114 matched prompts (means).

orthographic views at 25° elevation and eight azimuths (0° to 315° in 45° steps).

Text–image alignment. VQAScore (Lin et al. 2024) is the probability that Qwen2.5-VL-7B-Instruct (Bai et al. 2025) answers "Yes" to whether a rendered view shows the prompt. PE and SigLIP2 are image–text matching scores from PE-Core-bigG-14-448 (Bolya et al. 2025) and SigLIP2 giant-opt-patch16-384 (Tschannen et al. 2025), whose text encoders read at most 72 and 64 tokens, respectively. For each metric, a model's score is the maximum over its eight views, then averaged over prompts.

Invalid placements. For a model with placements $b_1, \ldots, b_n$ in exported file order,

$$P_{\text{inv}} = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left[b_i \text{ is invalid given } b_1, \ldots, b_{i-1} \right], \qquad (1)$$

where invalidity (e.g. intersection with earlier parts) is decided by BrickNet's original inset collision kernel (Kulits and Schmid 2026). We report the macro average over models. The original generation order is not verified, and a zero P_{inv} does not certify stud connectivity, stability, or buildability.

The systems differ in candidate selection, filtering, and post-processing; the comparison characterizes these delivered outputs rather than isolated model capability.

Metric	Raw	Retiled
$P_{\text{inv}} \downarrow$	0.1897%	0.0000%
VQAScore ↑	0.2065	0.2071
PE ↑	0.1858	0.1859
SigLIP2 ↑	0.1379	0.1261
Mean parts	178.7	421.2

Table 4: The 110 retiled outputs of the main comparison paired with their raw drafts (same prompt, model identifier, and caption).

5.2 Main Results

On these outputs, BrickMore has the highest mean on each alignment metric (Table 3). BrickMore and BrickGPT both have a mean P_{inv} of 0.00%, so this measure does not distinguish them. Per prompt, BrickMore scores higher than BrickGPT on 91, 95, and 91 of 114 prompts for VQAScore, PE, and SigLIP2, respectively; no significance test is reported. The mean part counts are 413.0 (BrickMore), 152.3 (BrickGPT), and 23.3 (BrickNet); these differences should be considered when interpreting the scores, and part count alone does not measure quality. Qualitative outputs are shown in Figures 1 and 3; historical qualitative material is in Appendix C.

5.3 Raw versus Retiled Outputs

For the 110 prompts whose main-comparison output is retiled, we pair the retiled layout with its raw draft (Table 4). Retiling reduces the mean invalid-placement ratio to zero and more than doubles the mean part count. VQAScore and PE change only slightly, while the mean SigLIP2 score decreases. These are descriptive results for this subset and do not establish a general causal effect.

5.4 Evaluation Boundaries

PE truncates 25 of the 114 prompts and SigLIP2 truncates 29 (mostly L4/L5); the affected prompts are the same across systems. Uniform yellow rendering means original colors are not assessed. The metric runs use Qwen2.5-VL-7B-Instruct for VQAScore and are not an exact reproduction of previously published evaluations, so the scores should not be compared directly with values reported elsewhere. VQAScore, PE, and SigLIP2 measure text–image alignment of renders; they are not a dedicated spatial-reasoning benchmark.

6 Limitations and Discussion

Scope. BrickMore learns spatial layout generation through data generation and supervised fine-tuning; it performs no pretraining or reinforcement learning, and its generator is a text-only language model. Vision–language models are used only as evaluation metrics on rendered images.

Data accounting. The integrated count of 79,301 models is derived from versioned release reports rather than a new full verification run. Dataset validation uses six-neighbour connectivity and permits several grounded brick components, whereas post-generation retiling enforces a single stud-connected component. Physical screening relies on stated assumptions and does not replace calibrated clutch-strength measurements or physical assembly tests.

Evaluation. The comparison covers 114 prompts and the provided outputs, which differ in candidate selection and post-processing, and some prompts are truncated by the metric models. These conditions limit how broadly the scores can be interpreted. Paired uncertainty analysis can be conducted on the existing per-prompt records.

7 Conclusion

We presented BrickMore, a text-to-brick framework that combines agentic batch generation of training data, language-model fine-tuning, and post-generation geometric repair. The dataset workflow converts programmatically generated voxel shapes into brick layouts, and the generation pipeline predicts placements from text and checks reconstructed layouts for coverage, overlaps, and stud connectivity. On 114 shared prompts, the evaluated BrickMore outputs achieve the highest means on three text–image metrics while matching BrickGPT on mean invalid-placement ratio. The results should be interpreted within the stated evaluation and physical-screening boundaries.

References

Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; et al. 2025. Qwen2.5-VL Technical Report. arXiv:2502.13923.

Bolya, D.; Huang, P.-Y.; Sun, P.; Cho, J. H.; Madotto, A.; Wei, C.; Ma, T.; Zhi, J.; et al. 2025. Perception Encoder: The Best Visual Embeddings Are Not at the Output of the Network. arXiv:2504.13181.

Chung, H.; Kim, J.; Knyazev, B.; Lee, J.; Taylor, G. W.; Park, J.; and Cho, M. 2021. Brick-by-Brick: Combinatorial Construction with Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*.

Kulits, P.; and Schmid, C. 2026. BrickNet: Graph-Backed Generative Brick Assembly. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. ArXiv:2604.22984.

Lin, Z.; Pathak, D.; Li, B.; Li, J.; Xia, X.; Neubig, G.; Zhang, P.; and Ramanan, D. 2024. Evaluating Text-to-Visual Generation with Image-to-Text Generation. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

Liu, R.; Deng, K.; Wang, Z.; and Liu, C. 2024. StableLego: Stability Analysis of Block Stacking Assembly. *IEEE Robotics and Automation Letters*, 9(11): 9383–9390.

Poole, B.; Jain, A.; Barron, J. T.; and Mildenhall, B. 2023. DreamFusion: Text-to-3D Using 2D Diffusion. In *International Conference on Learning Representations (ICLR)*.

Pun, A.; Deng, K.; Liu, R.; Ramanan, D.; Liu, C.; and Zhu, J.-Y. 2025. Generating Physically Stable and Buildable Brick Structures from Text. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. ArXiv:2505.05469.

Raistrick, A.; Lipson, L.; Ma, Z.; Mei, L.; Wang, M.; Zuo, Y.; Kayan, K.; Wen, H.; et al. 2023. Infinite Photorealistic Worlds Using Procedural Generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

Thompson, R.; Ghalebi, E.; DeVries, T.; and Taylor, G. W. 2020. Building LEGO Using Deep Generative Models of Graphs. arXiv:2012.11543.

Tschannen, M.; Gritsenko, A.; Wang, X.; Naeem, M. F.; Alabdulmohsin, I.; Parthasarathy, N.; Evans, T.; Beyer, L.; et al. 2025. SigLIP 2: Multilingual Vision-Language Encoders with Improved Semantic Understanding, Localization, and Dense Features. arXiv:2502.14786.

Walsman, A.; Zhang, M.; Kotar, K.; Desingh, K.; Farhadi, A.; and Fox, D. 2022. Break and Make: Interactive Structural Understanding Using LEGO Bricks. In *Proceedings of the European Conference on Computer Vision (ECCV)*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; et al. 2025. Qwen3 Technical Report. arXiv:2505.09388.

A Technical Appendix: Dataset Details

The figures and tables in this appendix are retained from the supplied reference material. Several figures still contain Chinese labels and use historical versions or criteria; they are not substitutes for the integrated statistics or the current main comparison. Values, physical-screening coverage, and missing entries require verification.



Figure 4: Voxel model samples spanning the 21 object categories.

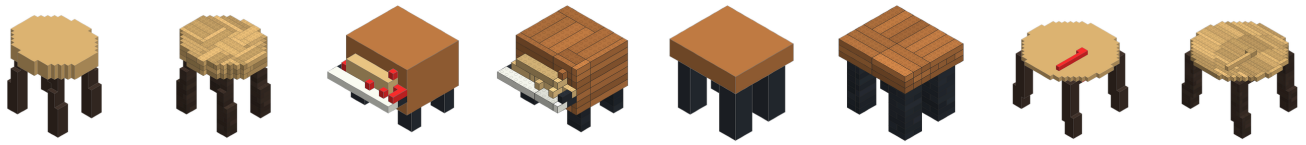


Figure 5: Historical examples of voxel targets and brick layouts after local repair.

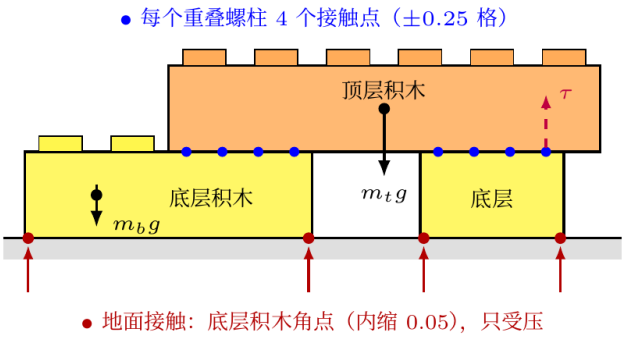


Figure 6: Historical contact-based gravity screening model (source labels in Chinese are retained).

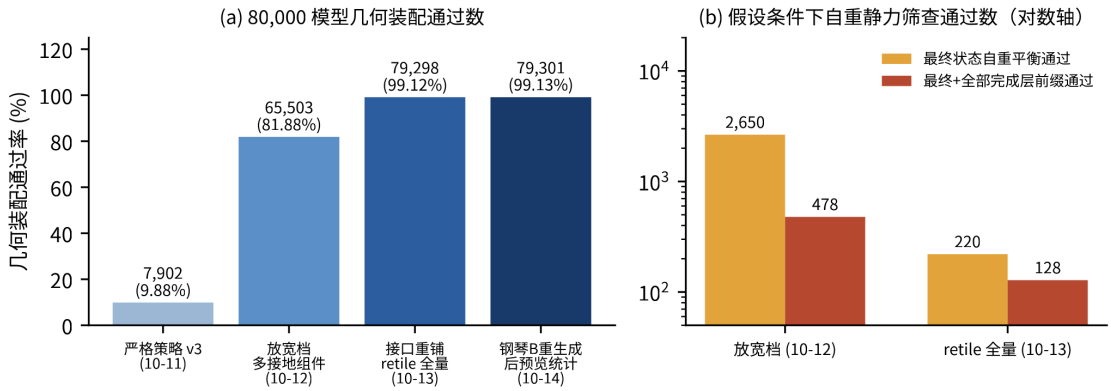


Figure 7: Historical comparison of geometric conversion and screening outcomes across releases. Version and numerical labels require verification.

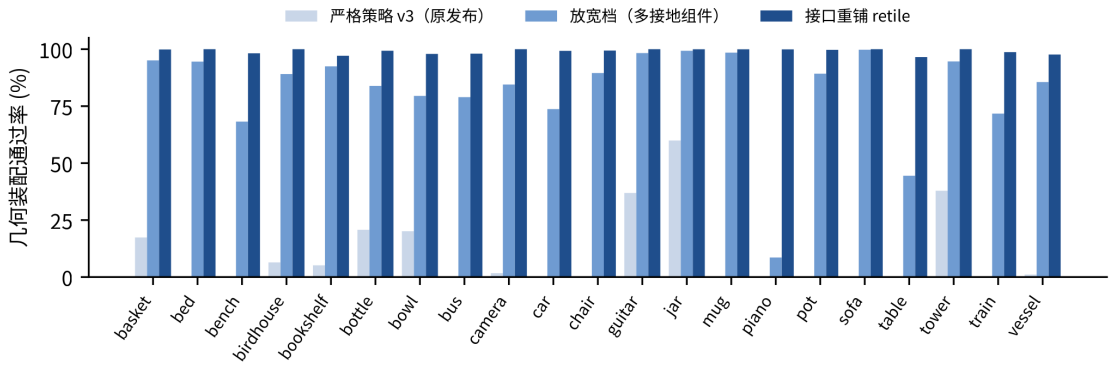


Figure 8: Historical category-level geometric pass rates. Version and numerical labels require verification.

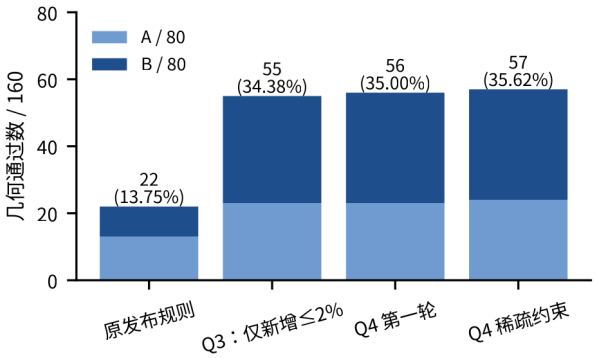


Figure 9: Historical results on a uniform 160-model sample. Version and numerical labels require verification.

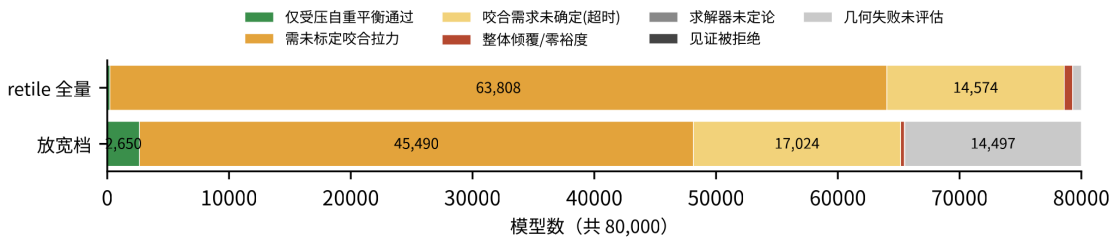


Figure 10: Historical distributions of physical screening statuses. Counts refer to the specific releases shown and are kept separate from geometric pass rates.

B Technical Appendix: Training Details

Aggregate throughput is 24,448 tokens/s over eight GPUs, and the mean training loss is 0.0273. Evaluation and checkpointing run every 500 steps. With TRL 0.15, the unpacked multi-GPU path hangs after the first backward pass, which is why the TRL 0.17 stack is pinned. Because validation loss is lowest during epoch 2, two epochs would be a more economical setting for future runs (about one third less compute). The full software stack is transformers 4.51.3, trl 0.17.0, peft 0.15.2, datasets 3.6.0, and accelerate 1.6.0, with bf16 precision, gradient checkpointing, and unpacked sequences.

Progress	Step	Validation loss
Start	500	0.1410
—	1,500	0.0950
—	10,500	0.0509
—	22,500	0.0404
$\approx$ End of epoch 1	44,500	0.0337
—	65,500	0.0316
Best (epoch 1.77)	79,000	0.0308
End of epoch 3	133,860	0.0339

Table 5: Validation loss on the 2,000-sample validation subset. One epoch is $\approx$44,620 steps; the nearest evaluation to the end of epoch 1 is step 44,500.

Dimension	BrickGPT paper	Official Qwen3 script (ours)
Base model	LLaMA-3.2-1B-Instruct	Qwen3-1.7B
LoRA	$r = 32$, $\alpha = 16$, q/v only (3.4M params)	$r = 16$, $\alpha = 32$, all linear layers
Learning rate	2×10^{-3} (AdamW)	10^{-4}
Effective batch	64	8
Max length	4,096	8,192
Epochs	3	3
Hardware	8×A6000, 12 h	8×4090D, 25.3 h
Data	StableText2Brick, 47k structures / 240k prompts	BrickMore-48, 79.3k models / 396,610 samples

Table 6: Our configuration versus the original BrickGPT paper. The data organization, instruction template, absence of thinking mode, and LoRA paradigm follow BrickGPT; concrete hyperparameters follow the official Qwen3 adaptation, with the budget adjusted for the larger world, library, and corpus.

C Technical Appendix: Additional Evaluation Material

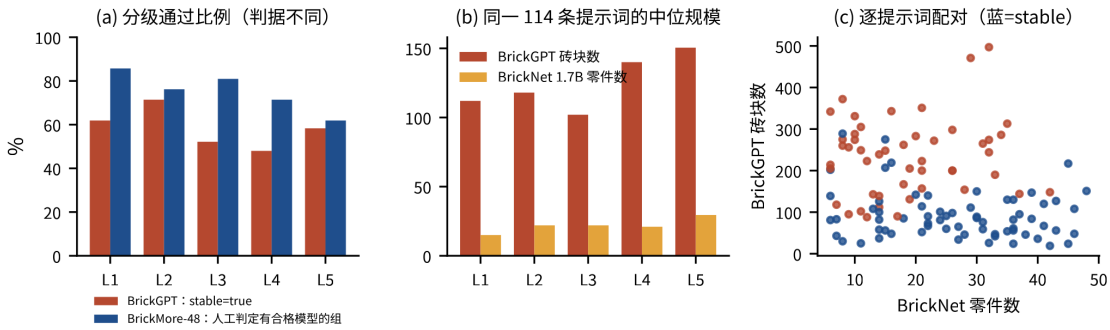


Figure 11: Historical baseline comparison with differing acceptance criteria; not the current matched main comparison.

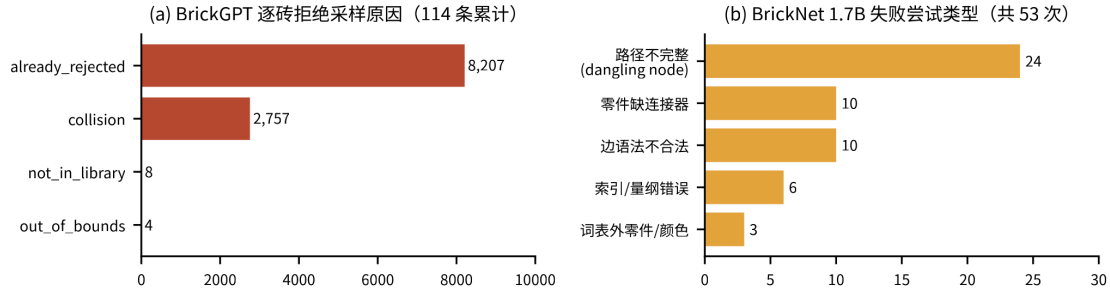


Figure 12: Historical failure categories in the supplied reference; sample definitions require verification.

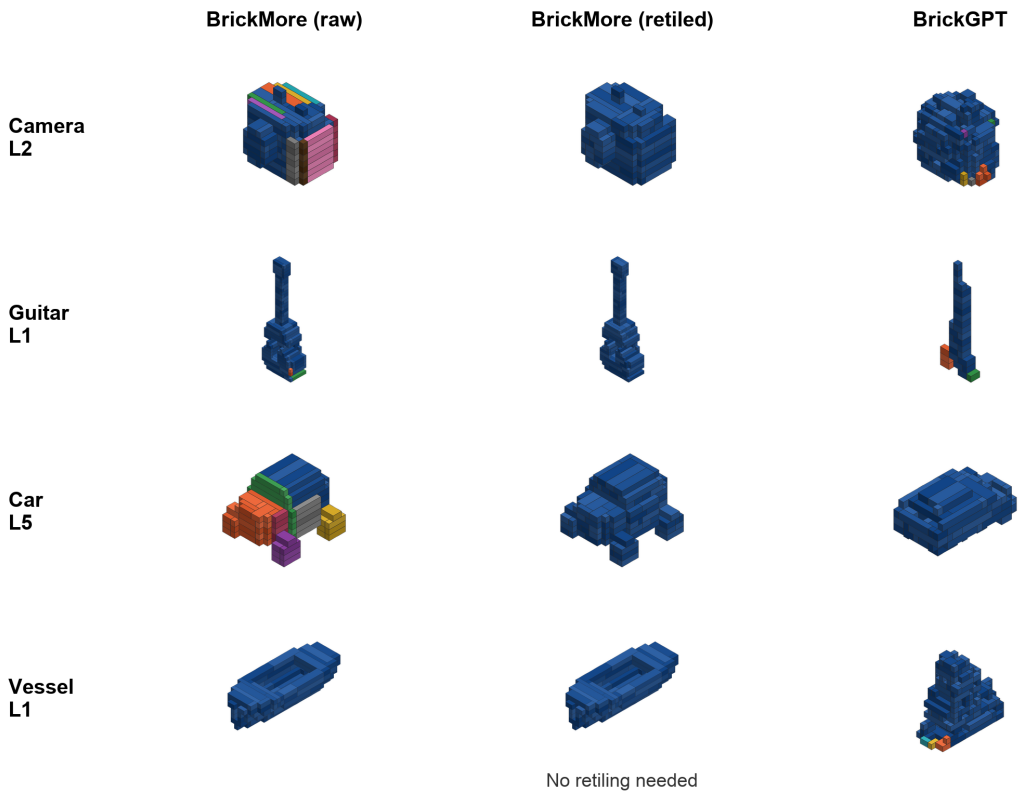


Figure 13: Qualitative historical examples from the supplied reference. Column labels distinguish raw BrickMore, retiled BrickMore, and BrickGPT outputs.

D Technical Appendix: Historical Tables

The following transcribed tables preserve the historical reference material. They use different versions or evaluation criteria and are not substitutes for the current main comparison. Dashes indicate missing entries in the supplied source.

Item	Historical value
Voxel models	80,000 (40,000 per batch)
Categories / structural families	21 / 40 per category
Grid / voxel count	48^3 / 750–1,250
Prompt records	400,000 (five bilingual records per model)
Uniqueness	80,000 unique; 0 A/B duplicates
Overlap with historical hashes	0 / 55,255
Retile geometry passes	79,298 (99.12%)
Bricks in full preview	12,718,389
Final gravity equilibrium passes	220
Final and layer-prefix passes	128

Table 7: Historical dataset statistics transcribed from the supplied reference. The geometry count refers to the old retile release, not the later integrated count. Verify the preview and physical-screening totals against version-matched reports.

Version	Connectivity	Voxel changes	Passes	Rate	Final	Final + prefixes
Strict v3	Single; support $\geq$10%, overhang $\leq$2	None	7,902	9.88%	–	–
Relaxed	Each component grounded	+428,278 / -38,230	65,503	81.88%	2,650	478
Retile	Each component grounded	+228,582 / -125	79,298	99.12%	220	128
Uniform 160-model sample (seed 38177); not full-dataset results						
Baseline	Single stud-connected component	None	22	13.75%	–	–
Q3	Single stud-connected component	Added $\leq$2%	55	34.38%	8	2
Q4 (sparse)	Single stud-connected component	Added + removed $\leq$2%	57	35.63%	8	2

Table 8: Historical conversion-strategy table from the reference. Full-dataset and 160-model sample rows use different scopes; connectivity rules also differ. Final and prefix columns report screening outcomes under the stated historical assumptions.

Item	BrickGPT	BrickNet 1.7B	BrickMore-48
Representation	20^3 bricks	Assembly graph	48^3 bricks
Criterion	stable	Parseable	–
Passes	66/114	114/114	–
Median parts	126.5	22	–
Range	19–497	6–48	–
Mean time (s)	189.6	–	–

Table 9: Historical baseline table in the source reference. These criteria differ, and missing BrickMore entries were marked TODO. This is not the current matched four-metric main table.

Level	n	BrickGPT stable	BrickGPT median bricks	BrickNet median parts	BrickMore-48
L1	21	13	112	15	–
L2	21	15	118	22	–
L3	23	12	102	22	–
L4	25	12	140	21	–
L5	24	14	150.5	29.5	–

Table 10: Historical level-wise baseline table transcribed from the reference. The original BrickMore column was incomplete, and the other columns use different criteria.

Method and source	P_{inv}	VQA	PE	SigLIP2
BrickGPT (literature, as transcribed)	0.063	0.050	0.157	0.052
BrickNet 1.7B (literature, as transcribed)	0.260	0.593	0.282	0.631
BrickGPT (114 prompts, historical placeholder)	–	–	–	–
BrickNet (114 prompts, historical placeholder)	–	–	–	–
BrickMore-48 (114 prompts, historical placeholder)	–	–	–	–

Table 11: Original reference table with literature values and historical placeholders. Literature values use another evaluation set and must not be directly compared with the present 114-prompt results; the precise P_{inv} definition and direction require checking against the original publication.